



TTK4147 Real-time Systems

05 Deadlocks, Mixed Criticality, and FreeRTOS

17.09.2025

1

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

 NTNU

1

Today

- Deadlocks
- Mixed Criticality
- FreeRTOS
 - Task Model
 - Scheduling
 - Inter-process Communication in FreeRTOS

2

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

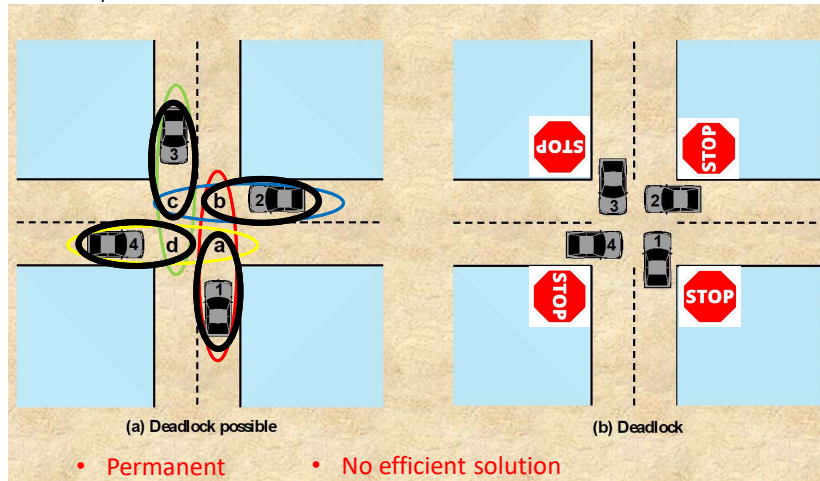
 NTNU

2

Deadlock and Starvation

What is «Deadlock»?

- Deadlock is when multiple processes are blocking each other.
 - Most cases this will be due to **shared resources** that several of the processes want to access at **the same time**.



3

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



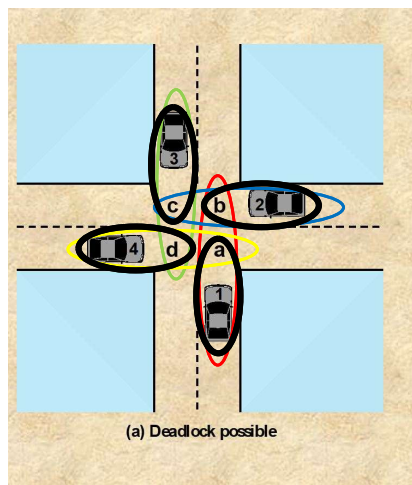
3

Deadlock, a question about concurrency

If sequential execution and non-preemptive, no problem

Remember the «Race condition» in lecture 03

A question about who is requesting the shared resources at which moment of time.



Thread 1	Thread 2	Integer value
read value	--	0
increase value	--	0
write back	--	0
read value	--	1
increase value	--	1
write back	--	1
read value	--	2
increase value	--	2
write back	--	2

- Car 1 and 3 are independent
- If car 4 does not exist, it would be just a question of time

4

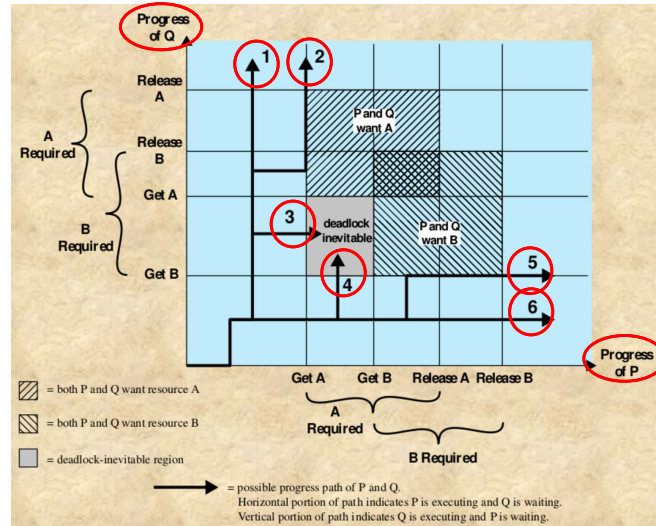
TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



4

Example of Deadlock, timeline analysis

Two processes, P and Q, which both need the resources A and B



5

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

5

Resource Types

- Reusable resources.

- Variable in memory.
- File in filesystem.
- Available memory

In total
200kB
memory

Process 1	Process 2
Request 80 kB	Request 70 kB
...	...
Request 60 kB	Request 80 kB
...	...
Free 60 kB	Free 80 kB
...	...
Free 80 kB	Free 70 kB

- Consumable resources.

- Produced then consumed, typically by different processes.

Blocked
receive

Process 1	Process 2
Receive from P2	Receive from P1
...	...
Send to P2	Send to P1



What can prevent
deadlock?

Key words: What happen concurrently?

6

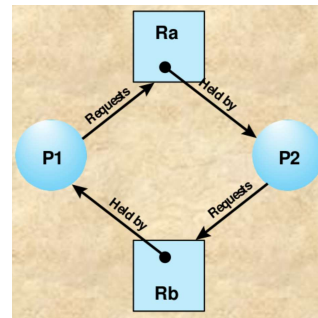
TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

6

Conditions for Deadlock

- Three conditions have to be present for a deadlock to be possible.
 1. Mutual Exclusion
 - Only one process can use a resource at a time (i.e. a printer).
 2. Hold and wait
 - A process can hold one resource (A) while waiting for another resource (B), while another process holds one resource (B) while waiting for another (A).
 3. No preemption of a resource
 - No resource can (forcibly) be removed / released from the processes holding it.
- For a deadlock to **actually** take place, a circular wait has to happen («4th» condition).
 - Circular wait is not a requirement for a deadlock, but a possible consequence when all three previous conditions are met.



7

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

7

Dealing with Deadlocks

3 strategies to deal with deadlocks:

- Deadlock Prevention
 - Adopt a policy that eliminates one of the conditions
- Deadlock Detection
 - Attempt to detect the presence of deadlock and take action to recover
- Deadlock Avoidance
 - Make the appropriate dynamic choices based on the current state of resource allocation

8

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

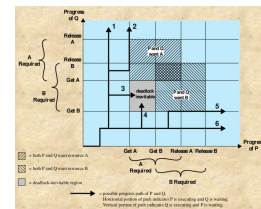
NTNU

8

Deadlock Prevention

Prevent deadlock by removing the necessary conditions.

- Indirect, by removing one of the three conditions that make deadlocks possible.
 - No mutual exclusive
 - In most cases it is not practical to remove this condition.
 - No hold and wait
 - Can be removed by forcing a thread to request all resources at the same time.
 - Unnecessary locking of resources.
 - Not always possible to know which resources are needed in the future.
 - Allow preemption
 - Allow the OS to force a thread to release its resource.
 - Not always possible, resource can be left in an unpredictable state.
- Direct, by removing the possibility of circular wait.
 - Define an order in which the resources must be requested. If all processes request resources in the same order, a circular wait is impossible.
 - Unnecessary locking of resources.



9

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



9

Deadlock Detection

- The deadlock detection method has no limitations for how processes should access resources.
 - The system will instead periodically check if any deadlock has occurred.
- If a deadlock is detected, the system has to be recovered.
- Recovery strategies.

Process related (present 3 alternatives):

- a) Abort all deadlocked processes, then restart them

What is the drawback? Deadlock may recur.



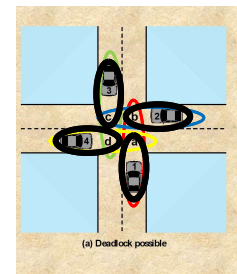
- b) Return each deadlocked process to a previous checkpoint.

- Will not always be possible.
- Also possible that the deadlock will recur.

- c) Abort deadlock threads one by one until the deadlock no longer exist, then restart them. Order of aborting can be decided by what is considered the smallest cost.

Resource related:

- a) Preempt resources until the deadlock no longer exist.



10

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



10

Deadlock Avoidance

More attractive than deadlock prevention, and it does not allow for deadlock to occur as deadlock detection does.

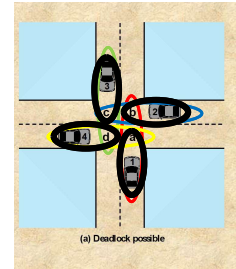
a) Process initiation rejection.

- Prevent (postpone) a process to start if it is likely to cause a deadlock. (I.e. car 4)

How to know?

- Check that the maximum use of resources from all existing processes and the process that wants to start are less than the available resources,
OR
Check if the new process needs a resource that is already taken

Not optimal, as it assumes the worst. (I.e. car 2 vs car 1)



b) Resource allocation rejection.

- Only allow a process to request a resource if that does not lead to an *unsafe state* that could cause deadlock.

Not optimal, as it assumes the worst.



What does this mean?

Risks to be too restrictive, as the worst may not, likely not happen

11

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

Summary of Deadlock Detection, Prevention, and Avoidance

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercommits resources	Requesting all resources at once	- Works well for processes that perform a single burst of activity - No preemption necessary	- Inefficient - Delays process initiation - Future resource requirements must be known by processes
		Preemption	Convenient when applied to resources whose state can be saved and restored easily	Preempts more often than necessary
		Resource ordering	- Feasible to enforce via compile-time checks - Needs no run-time computation since problem is solved in system design	Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	No preemption necessary	- Future resource requirements must be known by OS - Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	- Never delays process initiation - Facilitates online handling	Inherent preemption losses

12

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

Worst-case Execution Time and Mixed Criticality

- Two topics were I will go a little bit beyond what it covered in Burns and Wellings chapter 11.
- I do not have any "textbook" sources on this.
 - The information in the next slides is the syllabus.
 - The slides are based on references given below, but these are NOT required reading. (Easy to search for if you want to.)

(We will use

Response-time: The time from a task starts execution to it is completed [Burns and Wellings])

References:

Wilhelm, R., Engblom, J., Ermedahl, A., Holsti, N., Thesing, S., Whalley, D., ... & Mueller, F. (2008). "The worst-case execution-time problem—overview of methods and survey of tools." *ACM Transactions on Embedded Computing Systems*

Vestal, S. (2007). "Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance." *In Real-Time Systems Symposium*

Burns, Alan, and Rob Davis. (2013) "Mixed criticality systems-a review." *Department of Computer Science, University of York, Tech. Rep*

13

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



13

Different Levels of Worst-case Execution Time, 1 of 2

(To show how difficult it is to find exact execution times ☺)

- For real-time tasks we are concerned with the worst-case execution time (WCET).
 - How do we know the time for sure?
 - Is this one a fixed «value» or a fixed gap?

Worst case response time of a task, when there are higher priority tasks that interfere.

R_i : Response time of task i

D_i : Deadline of task i

C_i : The computational time of task i

I_i : Interference from higher priority tasks

$$R_i \leq D_i$$

$$R_i = C_i + I_i$$

I_i may be other user processes or kernel processes

14

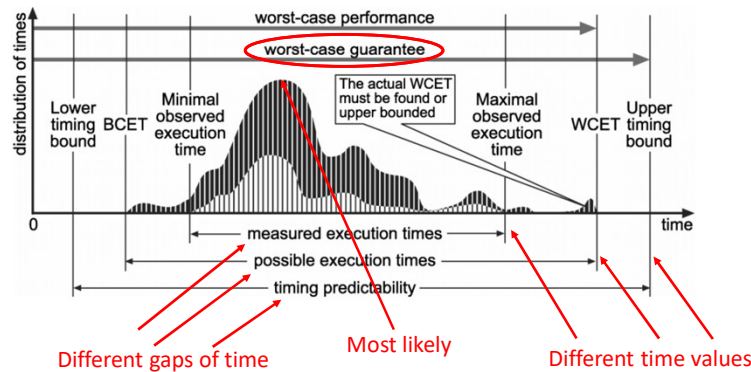
TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



14

Different Levels of Worst-case Execution Time, 2 of 2

- Different WCET levels from most pessimistic to most optimistic:
 - Highest theoretical, provable execution time, found with static analysis.
The actual value, but unfortunately difficult/impossible to get.
 - Histogram: The worst execution time found when testing x number of times.



15

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

15

Mixed Criticality

(Similar to "hard and soft task" as described in Burns and Wellings chapter 11.)

- Criticality is typically used as a description of the importance that a task is performed within a deadline.
 - In the simplest form binary; Critical or non-critical.
However, possibly more than two levels (although dual-criticality is often used).
- Example of three level mixed criticality:
 - Safety critical
 - Mission critical
 - Non critical
(Non critical tasks can be of hard real-time, firm real-time or soft real-time category)

16

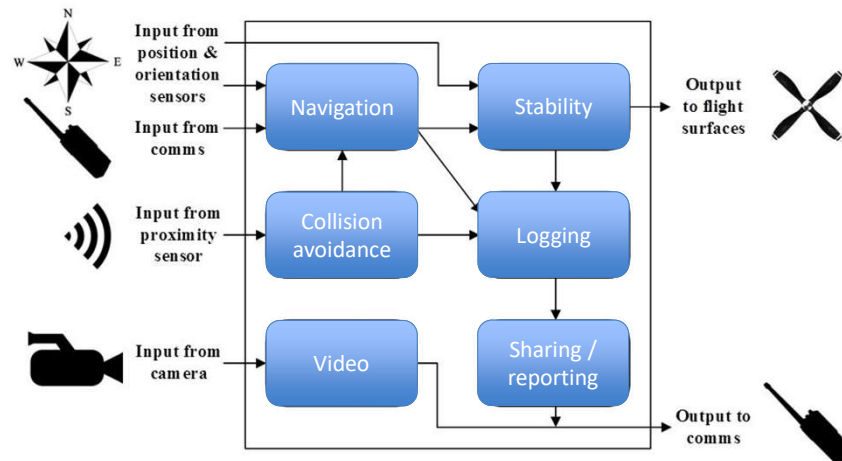
TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

16

Mixed Criticalty Unmanned Aerial Vehicle (UAV) Example:

Analyze the specific case, what is Safety critical, Mission critical, Non critical



17

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



17

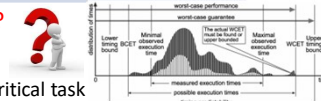
Dual Criticality UAV Example

Criticality	Max Observed WCET	Max Theoretical WCET
Safety	60%	90%
Mission	40%	-
Total	100%	90%

No margin. Is this acceptable?

• Perform a Response time analysis

- You should always be able to do the safety critical task
- If safety critical task takes more than the max observed (safety) load, then there might not be enough time for mission critical tasks.



• About verification / approval of the code - requirements

- The safety critical tasks might be subject to 3rd party approval (which may not find max observed WCET = 60% acceptable).
- The mission critical tasks will typically be approved internally.
 - Acceptable to have a *lower standard*, since your product will not be illegal if these tasks fail.

18

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

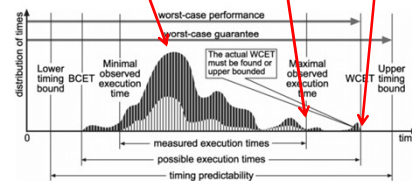


18

Tripartite mixed Criticality UAV Example

Criticality	Mean Execution Time	Max Observed WCET	Max Theoretical WCET
Safety	50%	60%	90%
Mission	30%	40%	-
Non	20%	-	-
Total	100%	100%	90%

- If all tasks perform at mean execution time, you will even be able to do the non critical tasks.
- Otherwise as before



19

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



19

Mixed Criticality; Guideline for scheduling

Criticality	Mean Execution Time	Max Observed WCET	Max Theoretical WCET
Safety	50%	60%	90%
Mission	30%	40%	-
Non	20%	-	-
Total	100%	100%	90%

1. All tasks should be schedulable using average execution times and average arrival rates.
2. All critical (Safety & Mission) tasks should be schedulable using worst case execution time and worst case arrival rates of all tasks (including non-critical).

20

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



20

Mixed Criticality; Guideline for scheduling continue

Criticality	Mean Execution Time	Max Observed WCET	Max Theoretical WCET
Safety	50%	60%	90%
Mission	30%	40%	-
Non	20%	-	-
Total	100%	100%	90%

1. All tasks should be schedulable using average execution times and average arrival rates.
 2. All critical (Safety & Mission) tasks should be schedulable using worst case execution time and worst case arrival rates of all tasks (including non-critical).
- A consequence of Point 1 is that there may be situations in which it is not possible to meet all current deadlines.
This condition is known as a transient overload.
 - Point 2 ensures that no critical task will miss its deadline.
If this gives rise to unacceptably low utilizations for “normal execution”, direct action should be taken to try and reduce the worst case execution times (or arrival rates).

NB Low criticality doesn't mean low frequency

21

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

21

Mixed Criticality; Example 1000 autonomous cars

1. Task A: Read LiDAR measurement
2. Task B: Read GPS measurement
3. Task C: Estimate its position based on measurements
4. Task D: Find the optimal route to the destination
5. Task E: Drive according to the plan
6. Task F: Avoid collisions and give way to pedestrians
7. Task G: Coordinate with other cars
8. Task H: Run the cars infotainment system (input and output)

High criticality: ???
Medium criticality: ???
Low criticality: ???

Why do we categorize the tasks?



Have to downgrade some tasks. If all are of high criticality, no one is more important and thus will not be assigned higher priority than other.

If all are of high criticality → needs larger CPU → what about normal utilization?

22

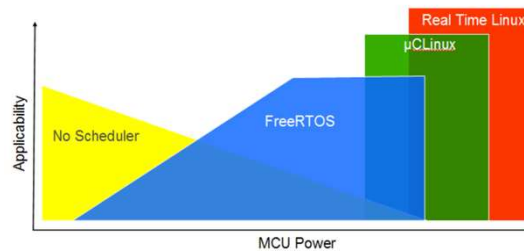
TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS

NTNU

22

Small OS : FreeRTOS

- Very small operating system
 - Consists of three main code files.
 - Plus architecture specific code.
- Usable for small micro-controllers.
 - Ported to more than 40 embedded system architectures.
 - Footprint from 9KB ROM.
 - ~200 bytes RAM and upwards.
- Very different from a "typical" operating system.



23

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



23

FreeRTOS Licensing

- Open and free under MIT open source license
- OpenRTOS: Same code-base, commercial license.
- SafeRTOS: Safety-certified version, commercial license

FreeRTOS

No support, except for online communities
No warranty

OpenRTOS

Support
Warranty

SafeRTOS

Support
Warranty
Certified to various safety standards

24

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



24

FreeRTOS Tasks

- Tasks have their own context. No dependency on other tasks unless defined.
- One task executes at a time.
- Tasks have no knowledge of scheduler activity. The scheduler handles context switching.
- Prioritized
- Preemptive.



Simple.



No restrictions on use.



Supports full preemption.



Fully prioritised.



Each task maintains its own stack resulting in higher RAM usage.



Re-entrancy must be carefully considered if using preemption.

25

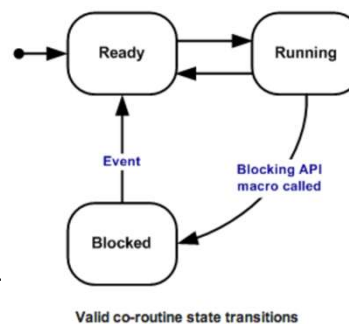
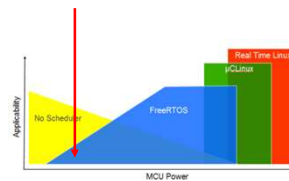
TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



25

FreeRTOS Co-Routines

- Intended for use on small processors that have severe RAM constraints.
- Simple state model
- Share a single stack.
- Prioritized relative to other co-routines, can be included in an application that uses preemptive tasks.
- The structure of co-routines is rigid due to the unconventional implementation.
 - Lack of stack requires special consideration.
 - Restrictions on where API calls can be made.
 - Cannot communicate with tasks.



26

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



26

FreeRTOS Co-Routine summary

- 😊 Sharing a stack between co-routines results in much lower RAM usage.
- 😊 Cooperative operation makes re-entrancy less of an issue.
- 😊 Very portable across architectures.
- 😐 Fully prioritised relative to other co-routines, but can always be preempted by tasks if the two are mixed.
- 😞 Lack of stack requires special consideration.
- 😞 Restrictions on where API calls can be made.
- 😞 Co-operative operation only amongst co-routines themselves.

Cooperative:
Voluntarily gives up its running state / blocks itself

27

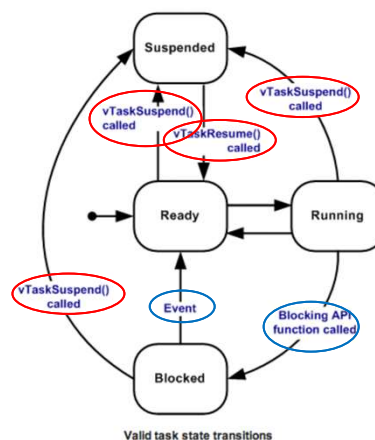
TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



27

FreeRTOS Task Model

- **Running:**
Actively executing and using the processor.
- **Ready:**
Able to execute, but doesn't because a task of equal or higher priority is in the Running state.
- **Blocked**
Waiting for a temporal or external event. E.g., queue and semaphore events, or calling `vTaskDelay()` to block until delay period has expired. Always have a "timeout" period, after which the task is unblocked.
- **Suspended**
Only enter via `vTaskSuspend()`, depart via `xTaskResume()` API calls.



28

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



28

Task Priorities

- Each task gets a priority from 0 to configMAX_PRIORITIES – 1
- Priority can be set on a per application basis.
- Tasks can change their own priority, as well as the priority of other tasks.

29

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



29

The Idle Task

- The idle task is created automatically when the scheduler is started.
- It frees memory allocated by the RTOS to tasks that have since been deleted.
 - Will only run when nothing else is needed, thus this can take some time.
 - Thus, applications that use vTaskDelete() to remove tasks should ensure the idle task is not starved.
 - The idle task has no other function, so cases when the idle task need never run exist.
- There is an idle task hook, which can do some work at each idle interval

30

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



30

FreeRTOS Scheduling

- Can be either co-operative or preemptive.
- Co-operative means non-preemptive
 - Each task runs until:
 - It is finished
 - It is blocked
 - It voluntarily "yields" its running status, allowing the scheduler to pick the highest priority task.
- The scheduler is activated by a timer interrupt every "clock tick".
 - If preemptive, the scheduler will check for any ready tasks that should be allowed to run and starts a context switch if any.
(If non-preemptive, the timer interrupt routine only increments the tick count.)
- The scheduler will select one of the ready tasks with the highest priority.
 - Equal priority tasks are scheduled with Round-Robin.

31

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



31

Inter-process communication in FreeRTOS

- FreeRTOS have implemented common inter-process communication methods.
 - Queues (same as message passing).
 - Semaphores (both binary and counting).
 - Mutexes (priority inheritance and recursive).
- FreeRTOS also have their "own" *task notification* concept.
 - Supposed to be 45% faster and use less RAM than binary semaphore.
 - Can be used as light-weight alternatives to other inter-process communication methods, with some limitations.
 - Each task has its 32 bit notification value.
 - An RTOS task notification is an event sent directly to a task that can unblock the receiving task, and optionally update the receiving task's notification value.
 - Can be used as a binary or counting semaphore or a queue

32

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



32

Queue Overview

- Queues are the primary form of inter-task communications.
(Used in implementation of semaphores and mutexes)
- They can send messages between tasks as well as between interrupts and tasks.
- Supports
 - appending data to the back of a queue
 - sending data to the head of a queue.
- Queues can hold arbitrary items of a fixed size.
- The size of each item and the capacity of the queue are defined when the queue is created.
- Items are enqueued by copy, not reference.

33

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



33

Queues and Blocking

- Access (read / write) to queues is either blocking or non-blocking:
 - The scheduler blocks tasks when they attempt to read from or write to a queue that is either empty or full, respectively.
 - If the xTicksToWait variable is zero and the queue is empty (full), the task does not block. Otherwise, the task will block for xTicksToWait scheduler ticks or until an event on the queue frees up the resource.
- This includes attempts to obtain semaphores, since they are special cases of queues.

34

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



34

Binary Semaphores

- Used for both mutual exclusion (protects critical regions) and synchronization.
- Commonly used by Interrupt Service Routines (ISRs) to wake tasks and avoid polling.

Consider the following example.

- A binary semaphore is created along with a task that blocks on this semaphore.
- An ISR is written for a peripheral that sets the semaphore when the peripheral requires servicing using `xSemaphoreGiveFromISR()`.
- This awakens the task waiting on this semaphore. The task resets the semaphore, does some work, and then blocks again.

35

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



35

Counting Semaphores and Mutexes

Counting Semaphores

- FreeRTOS has support for counting semaphores, the standard down and up (wait and signal) operations.
- Semaphores are macros defined over the queue API. Therefore, semaphores use the same API calls as queues do.

Mutexes

- FreeRTOS supports mutexes (binary semaphores with priority inheritance).
- As mutexes use the semaphore API, they also support the blocking timeout mechanism. Moreover, they are implemented using the queue API calls.
- The queue data structures are protected from corruption by disabling interrupts in some places and disabling the scheduler in others where it would be okay for an interrupt to occur, but not for another task to preempt the queuing task. (critical regions)

36

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



36

Interesting information about FreeRTOS

37

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



37

ROS on FreeRTOS

- The Robot Operating System (ROS) is an open source software framework for robotics applications development.
- ROS 2 is the second generation of ROS
- micro-ROS is ROS 2's younger sibling, that offers most of the appealing tools and functionalities of the fully deployed ROS 2 ecosystem
- FreeRTOS has been one of the first RTOSes to be supported by the micro-ROS project
- FreeRTOS has two kind of tasks: standard and idle tasks.
- A micro-ROS application is integrated into the FreeRTOS as a standard task with a given priority
- A future action planned by the micro-ROS project is that of adopting the certified version of FreeRTOS, SafeRTOS.

38

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



38

FreeRTOS support

Easy-to-follow web pages (<https://www.freertos.org/>) with information and downloads about:

- **Kernel**
 - How-to-start instructions,
 - Documentation,
 - Developer documentation,
 - Supported devices (processor families and tools)
 - API reference
- **Libraries (modular)**
 - **FreeRTOS+** provide connectivity and utility functionality
 - **IoT libraries** provide connectivity and security libraries functionality
 - **FreeRTOS Labs** optimized parts for improve memory usage, modularity, documentation, demo usability, or test coverage

39

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



39

FreeRTOS support continue

- **Resources**
 - **Manuals and books**
 - **Windows Simulator.** FreeRTOS Windows Port For Visual Studio or Eclipse and MingW
 - **Posix/Linux Simulator** Demo for FreeRTOS using GCC
- **Community**
 - **Forums**
 - **Blog**
 - Ecosystem, constantly growing add-on and complementary products such as:
 - **SafeRTOS**, Functional safety pre-certified RTOS for embedded processors
 - **OpenRTOS**, Commercially licensed version of the FreeRTOS kernel
 - **Fail Safe File System**, True fail safe transactional file system protects critical data
 - **Tracealyzer**, Get 20+ graphically interlinked views of the trace
 - **WolfSSL**, Networking security libraries (SSL/TLS) for embedded systems
 - 3rd Party **Board Support Packages (BSP)**, that include pre-integrated software packages (Atmel, Future Design Inc, NXP Semiconductors, STMicroelectronics)

40

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



40



41

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



41

Endings

2020-21

What are the four conditions for deadlock?

What conditions need to be satisfied for a deadlock to occur? Name the conditions and describe each with one sentence.

- Mutual exclusion: only one task can have access to a given resource at a time.
- Hold and wait: each task takes the free resources and waits for the blocked ones to be released.
- No preemption: a resource can only voluntarily be released by a process.
- Circular wait: processes must wait for resources being held by other resources in a circular pattern, so that they are blocking one another.

Only the first three have to be present for a deadlock to be possible, the last needs to take place for it to happen.

42

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



42

0000

Give some strategies to deal with deadlocks and give a one-sentence explanation to each.

- Deadlock Prevention
Adopt a policy that eliminates one of the conditions
- Deadlock Detection
Attempt to detect the presence of deadlock and take action to recover
- Deadlock Avoidance
Make the appropriate dynamic choices based on the current state of resource allocation

43

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



43

2018-10

You are part of a team designing the software system for a fixed-wing unmanned aerial vehicle (a plane) with a camera mounted on a gimbal. Due to size, weight, and power constraints, all tasks in this system must be run on the same processor. You have come to the conclusion that the set of tasks that must be run are:

- A. Receiving radio messages whenever they arrive (signaled with an interrupt).
- B. A set of tasks for polling the position sensors (GPS, accelerometer, etc).
- C. A high-level navigation task, for following a predetermined path ("where to fly").
- D. A set of tasks for flight control and stabilization, controlling power to motor and control surfaces ("how to fly").
- E. Gimbal control, to control where the camera points.
- F. Live video preview, to ensure that the mission is going as planned (the video recording itself is handled in-camera, and is not run on this processor).
- G. Handling of fire alarm interrupts.

Explain how you would schedule these different tasks.

Organizes tasks into criticality levels (High, Medium, Low)

High: flight, sensors, fire. Mid: nav, radio. Low: video, gimbal)

Frequently scanned (=Periodically executed) / Event (=ISR) - driven

44

TTK4147 – 2025: Deadlocks, Mixed Criticality, and FreeRTOS



44